

useNavigate vs Link: useNavigate is a hook for programmatic navigation (e.g., after form submission), while Link is a component for declarative navigation (e.g., clickable menu links).

Dynamic Form Fields: Maintain an array in state mapping to input fields, with functions to add/remove items by updating the array, then render the fields dynamically using `array.map()`.

useNavigate vs Link with Examples:

useNavigate (Programmatic):

jsx

```
import { useNavigate } from 'react-router-dom';

function LoginForm() {
  const navigate = useNavigate();

  const handleSubmit = (e) => {
    e.preventDefault();
    // After successful login
    navigate('/dashboard');
  };

  return <form onSubmit={handleSubmit}>{/* form fields */}</form>;
}
```

Link (Declarative):

jsx

```
import { Link } from 'react-router-dom';

function Navbar() {
  return (
    <nav>
```

```
    <Link to="/">Home</Link>
    <Link to="/about">About</Link>
    <Link to="/contact">Contact</Link>
  </nav>
);
}
```

Dynamic Form Fields Implementation:

jsx

```
import React, { useState } from 'react';

function DynamicForm() {
  const [fields, setFields] = useState(['']);

  const addField = () => {
    setFields([...fields, '']);
  };

  const removeField = (index) => {
    setFields(fields.filter((_, i) => i !== index));
  };

  const handleChange = (index, value) => {
    const updatedFields = [...fields];
    updatedFields[index] = value;
    setFields(updatedFields);
  };

  return (
    <div>
      {fields.map((field, index) => (
        <div key={index}>
          <input
            type="text"
            value={field}
            onChange={(e) => handleChange(index, e.target.value)}
            placeholder={`Field ${index + 1}`}
          />
          <button onClick={() => removeField(index)}>Remove</button>
        </div>
      ))}
    </div>
  );
}
```

```
        </div>
    )})
    <button onClick={addField}>Add Field</button>
</div>
);
}
```